



Katja Dahlman

The Journey to Quality: Understanding and Improving Defect Reporting

Metropolia University of Applied Sciences

Bachelor of Engineering

Information Technology

Bachelor's Thesis

1 April 2025

Abstract

Author:	Katja Dahlman
Title:	The Journey to Quality: Understanding and Improving Defect Reporting
Number of Pages:	24 pages + 2 appendices
Date:	1 April 2025
Degree:	Bachelor of Engineering
Degree Programme:	Information Technology
Professional Major:	-
Supervisors:	Salonen Janne, Director, School of ICT

This thesis explores how to improve defect reporting, how it effects to overall quality of the software and the key components of a good defect report. The thesis also describes how Artificial Intelligence and human factors can improve defect reporting practices.

A good defect report is clear and comprehensive with keeping it concise. Well-structured defect report increase overall efficiency and quality, improves resolution time and reduces costs of development. The overall quality of the software is easier to track when defects are documented.

The findings also highlight that while AI provide tools for enhancing defect reports and the reporting process, human expertise is still essential for ensuring software quality. Communication, collaboration and empathy has a positive effect on both quality of the software and efficiency.

I have used OpenAI ChatGPT-4.o mini and Google's Gemini Advanced 2.0. Flash for brainstorming the research design, structuring the work, and creating titles. I have also used the same programs for refining the language of my text and formatting citations. Additionally, I have used these programs as a source for section 3.3. The discussions can be found in Appendices 1 and 2. As the author of this thesis, I am responsible for all its content.

Keywords:	defect, defect reporting, bug reporting, defect management, software quality, software testing, AI in defect reporting
-----------	--

The originality of this thesis has been checked using Turnitin Originality Check service.

Tiivistelmä

Tekijä: Katja Dahlman
Otsikko: Virheraportoinnin ymmärtäminen ja parantaminen
Sivumäärä: 24 sivua + 2 liitettä
Aika: 1.4.2025

Tutkinto: Insinööri (AMK)
Tutkinto-ohjelma: Tieto- ja viestintätekniikka
Ammatillinen pääaine: -
Ohjaajat: Osaamisaluejohtaja, ICT ja tuotantotalous
Janne Salonen

Insinööriyössä tutkittiin kuinka virheraportointia voi parantaa ja millaisia hyviä käytäntöjä virheraportoinnissa voi ottaa käyttöön sekä miten tämä vaikuttaa sovelluksen kokonaislaatuun. Työssä tutkittiin myös, miten tekoäly ja inhimilliset tekijät vaikuttavat virheraportointiin.

Hyvä virheraportti on selkeä ja kattava, mutta samalla ytimekäs. Hyvin jäsennelty ja laadittu virheraportti parantaa sovelluksen laatua, nopeuttaa virheen käsittelyä ja korjaamista sekä voi vähentää kehityskustannuksia.

Sovelluksen kokonaislaatua on myös helpompi seurata ja mitata kun viat ovat dokumentoitu ja vianhallinnassa on selkeä prosessi, joka tukee kehityksen prosesseja. Tekoäly tarjoaa työkaluja virheraporttien ja prosessin tehostamiseen, mutta tulokset korostavat myös, että ihmisen asiantuntemus on edelleen olennaista ohjelmiston laadun varmistamiseksi. Kommunikaatiolla, empatialla ja yhteistyöllä on positiivinen vaikutus sekä laatuun että työn tehokkuuteen.

Olen käyttänyt OpenAI ChatGPT-4.0 mini sekä Googlen Gemini Advanced 2.0. Flash tutkimusasetelman ideoinnissa, työn jäsentelyssä sekä otsikoinnissa. Olen käyttänyt samoja ohjelmia tekstini kieliasun viimeistelyyn ja lähdeviitteiden muotoiluun. Olen käyttänyt samoja ohjelmia kappaleen 3.3 lähteenä. Keskustelut löytyvät liitteestä 1 ja 2. Opinnäytetyön tekijänä olen vastuussa kaikesta opinnäytteeni sisällöstä.

Avainsanat: virheraportti, bugiraportti, sovelluksen laatu, sovellustestaus, tekoäly virheraportoinnissa

1	Introduction	6
2	Understanding Defect Reports and Their Role in Software Quality	7
2.1	The Importance of Defect Reporting in Software Development	7
2.2	Key Components of a Good Defect Report	9
2.3	Why Defect Report Management Matters	10
3	Enhancing Defect Reporting Effectiveness	12
3.1	Why Defect Report Quality Matters and Common Issues	12
3.2	How Data and AI Can Help with Defect Reporting	13
3.3	AI's Answer to Improving Quality of Defect Reports	16
4	The Human Factor in Defect Reporting	19
4.1	Effective Communication in Defect Reports	19
4.2	Collaboration and Feedback in Defect Reporting	19
4.3	The Role of Empathy in Defect Reporting	22
5	Conclusion	23
	References	25

Appendices

Appendix 1: Discussion with Google Gemini Advanced 2.0. Flash

Appendix 2: Discussion with ChatGTP-4.o mini

List of Abbreviations

- AI: Artificial Intelligence. It is a branch of computer science.
- ML: Machine Learning. A subfield of AI, concentrating on handling data and making predictions and learnings from this data.
- NLP: Natural Language Processing. Subfield of AI, concentrating on natural language processing and understanding.
- QA: Quality Assurance. The process of ensuring that the that quality requirements will be met.
- SDLC: Software Development Life Cycle. The set of processes that outline tasks required to build and maintain a software.

1 Introduction

Software is everywhere and an integral part of our everyday life. Most people use several different software daily. Ensuring high quality of software is necessary. Defect report play a crucial role ensuring this quality.

Defect reports are documents created in the defect managed process describing faults or unexpected behaviour that does not match specification. Defect reports are an essential tool to communicate defects to developers, testers and other stakeholders in software development process. They provide necessary information to identify, understand and resolve software defects. (Son H., 2024). However, defect reports can often vary in quality, having incomplete information, lacking clarity and consistency (Bettenburg N. et al, 2008). Defect management process can be inefficient. These challenges can slow down the defect resolution process, increase development costs and impact software quality. (Gromova, A. et al., 2019). This makes it essential to maintain and value high-quality defect reports.

The purpose of this thesis was to research and analyse ways how to improve quality of defect reports and explore the best practices of defect reporting. Chapters to follow will provide foundational knowledge of defect reporting and its process, discussing key components of defect reporting and explaining the importance of good defect management process. This will be followed by how to enhance defect reporting effectiveness and examining what are the common issues in defect reporting and how AI and data can be leveraged in defect reporting lifecycle. As this process very heavily done by people, human factor is introduced. In this part significance of effective communication, collaboration, feedback and empathy is introduced. This section will discuss how communication, collaboration, and empathy intersect with defect reporting lifecycle.

2 Understanding Defect Reports and Their Role in Software Quality

2.1 The Importance of Defect Reporting in Software Development

In modern life, it's almost impossible to avoid using and seeing software. From communication to finance and transportation, software is fundamental to modern society, making high software quality a crucial concern. For example, bus stop has a timetable showing the next bus, and an app to purchase tickets. The Post office has an app to inform about incoming packages. Banking is mostly handled by apps and via their websites. Even the Finnish Tax Office is attempting to encourage people to use their apps and online services through memes and social media.

Many people have experienced software not functioning as expected. Testing is essential to ensure the reliability, security, and usability of a software.

Software quality has also become a critical factor in a company's reputation. A single defect can lead to severe consequences. A recent example of this is the S-Bank data breach in 2022, which was caused by a faulty login component that allowed a small group of users to access other people's accounts. Alarmingly, one of the individuals who discovered this vulnerability reported it to S-Bank, yet it remained unaddressed for three months until it was exploited. (Nyman M., 2024) This case highlights the importance of having an effective defect reporting system and a structured process for handling reported defects. It demonstrates how a well-managed defect reporting process and well written defect report can prevent significant security risks and financial losses.

Software testing is often mixed with Quality assurance. These are distinct processes within the software development life cycle (SDLC). Quality assurance is more focused on processes and improving them to prevent defects from happening. It's a continuous process that starts in the beginning and continues throughout the entire software development lifecycle. (GeeksforGeeks, 2023). Ideally, quality assurance is a responsibility shared by all team members

involved in software development (ISTQB, 2024). By emphasizing preventive measures, companies can reduce the occurrence of defects and save time and resources in later testing stages. This proactive approach can minimise technical debt and enhance the overall sustainability of the software. (CQI, 2023).

Software testing is a set of activities to discover defects and evaluate the quality of software. (ISTQB, 2024, p.15).

There are many misconceptions of testing. Most common of those are that testing is just executing tests and verifying results. It's more than just executing test cases; it is a comprehensive effort to verify functionality, identify defects, and ensure compliance with requirements. (ISTQB, 2024).

Two critical testing concepts include verification and validation. Verification is ensuring that the software meets specified requirements. Validation is confirming that what has been specified will meet the needs of the user and stakeholders. (ISTQB, 2024). Testing plays a crucial role in quality assurance, helping detect and document defects effectively.

Defect reporting involves a process that can vary between companies, but the basics remain the same. Defect reports are crucial to SDLC as they provide important information about the quality and severity of possible issues. It's also easier to track the quality level of the product when defects are documented.

Well and thoughtfully written defect report will help determine severity of defect found, its impacts to software and its possible users, and they can prevent system failures. Developers will get detailed information from defect reports to diagnose the issue.

Key testing principles relating to defect reporting are "Testing shows the presence, not the absence of defects" (ISTQB, 2024, p.17) and "Defects cluster together." (ISTQB, 2024, p.18). In my personal experience, where you find one defect, another is not far away.

2.2 Key Components of a Good Defect Report

What is a good defect report? This will vary depending on the person who is asked. As testing is highly context-dependent, defect reports must align with company policies, project requirements, and industry standards. Despite these variations, there are key components that every well-structured defect report should contain.

A defect report should be clear, concise, and comprehensive. The language and terminology used must be consistent with company guidelines to ensure that anyone reading the report can understand the issue without additional explanations. (Son, H. 2024).

1. Title should be written clearly and describe the issue as well as possible for the reader to grasp the situation immediately from one sentence.
2. Description will provide detailed information about defects, including what went wrong and where.
3. Steps to reproduce should give a step-by-step guide how defects can be reproduced.
4. Expected result will outline what should have happened, and the actual result will inform what happened when the test was run.
5. Environment details will give information about version used, environment and relevant configurations for easier debugging.
6. Visual evidence such as logs, stack traces and screen shots should be added when possible.
7. Priorities should be used, when possible, for triaging and to determine urgency of how quickly an issue needs resolving. (Son, H. 2024)

Developer feedback highlights that a well-structured defect report significantly improves resolution time. (Bettenburg N. et al., 2008). Based on this feedback, most important information for developers in defect report are:

- completeness of information
- steps to reproduce the defect are correct and reproduceable
- expected vs. actual result is clearly stated
- test cases are attached to defect report
- version number of software under test
- avoiding duplicate reports
- stack traces if available

Studies show that well-written defect reports receive faster attention from developers compared to incomplete or vague reports. (Bettenburg N. et al., 2008).

2.3 Why Defect Report Management Matters

One major objective of defect reporting is to provide accurate and actionable insights into software quality (ISTQB CTAL-TM, 2024). Without a structured defect management system, tracking and resolving defects becomes difficult. With this in mind, the defect reporting process and system need to be implemented. All members of the team should be trained to use defect tracking systems efficiently. The implementation of the defect management system should follow defect reporting and fixing processes to provide a good workflow.

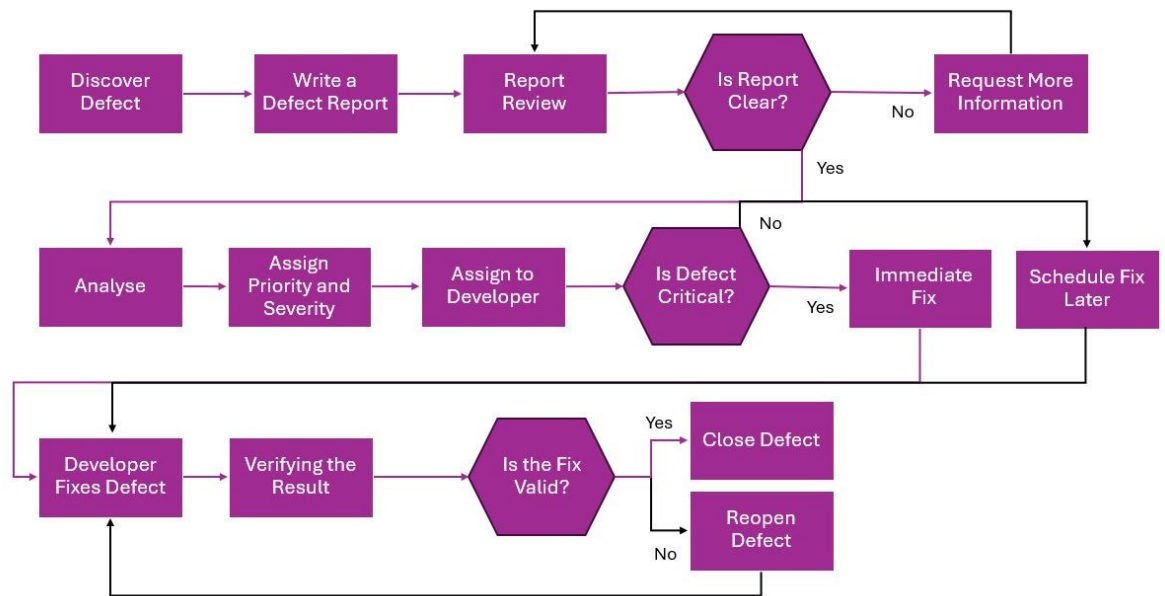


Figure 1. Simple process of defect management (Son, H. 2024).

Effective defect management ensures that defects are categorized and prioritized based on their severity and impact. It helps prevent unnecessary delays in the process of defect fixing, which includes analysing, fixing and testing the defect reported. Defect management process also helps balance workload distribution within the development team. (Son, H. 2024). Not tracking defects greatly reduces the visibility of quality status of the software and makes it difficult carry out improvements due to lack of reliable support data (ISTQB CTAL-TM, 2024).

During defect reporting other anomalies such as change requests, and false positives can be uncovered. These issues are resolved through structured defect management processes. Some defect tracking tools offer automation features, allowing systems to auto-populate certain report fields and ensure report consistency. (ISTQB, 2024).

Defect tracking systems and processes do improve software quality, but some challenges still exist. Missing details can cause delays on handling and debugging the defect in question. Duplicate defect reports may lead to waste of

work effort and distort the quality metrics. Improvement ideas reported as defects can also distort the quality metrics.

Defect tracking tools also provide valuable insights into recurring defect patterns, enabling teams to proactively improve software quality and efficiency. By effectively managing defect reports, teams can enhance software quality, improve defect resolution efficiency, and prevent similar issues from happening in future. This aligns with the testing principle that “Defects cluster together”. (ISTQB, 2024). By managing defect reports effectively, companies can enhance software reliability, streamline maintenance efforts, and reduce long-term costs.

3 Enhancing Defect Reporting Effectiveness

3.1 Why Defect Report Quality Matters and Common Issues

Quality matters across the development lifecycle. High quality defect reports will decrease the resolution time and increase overall probability of defects getting fixed. (Gromova, A. et al., 2019).

Defect reports relay information to developers and other stakeholders about how and what part of the software does not meet quality expectations, making them an important tool to understand what needs changing or fixing. Missing or misleading information in the defect report can lead to longer resolution time, inefficiency in defect management and longer maintenance cycles. It can also lead to incorrect prioritizing.

Correct prioritizing is crucial for the project, as early defect detection and fixing saves money and time. If a defect is discovered late, it is usually more costly to fix it too and can have more dependencies than if it would have been fixed earlier. This also impacts the overall maintainability and technical debt of the software. (Jahanshahi, H et al. 2022).

Defect reports are a great tool for measuring and monitoring quality and work effort in the project. Defect reports can be included into metrics to measure how

testing is progressing and if test objectives are met. They can be also used to estimate test effectiveness with calculating defect detection percentage (DDP). High DDP is usually strived for, as it indicates how much defects have been discovered before production compared to after production. Defect reports can also support process improvements during retrospectives. Defect report metrics can be used to find out in which area of software has largest cluster of defects to target improvement for defect prevention. They can be used to find root causes which can help reduce future defects. (ISTQB CTAL-TM, 2024).

Analysing defect reports and their effect to software quality helps improve overall quality in future, for example:

- Information from re-opened defects can be used to improve debugging.
- Information from duplicate and rejected defect report can be used to improve quality of defect reports.
- Information from defect clusters can be used to understand technical risks of the software.
- Information from defect reports can be used to improve performance of the software. (ISTQB CTAL-TM, 2024).

Keeping this in mind, structured information can improve defect tracking. Good concise header will improve categorization and prioritization. Well written defect report will improve resolution time, limit the need of asking for more information and prevent misclassification.

3.2 How Data and AI Can Help with Defect Reporting

Artificial Intelligence (AI) can be a helpful assistant in the defect reporting process. AI can enhance efficiency and accuracy through automation and predictive analysis.

So, what AI even is? There is not a commonly agreed definition of Artificial Intelligence. It is a branch of computer science and usually refers to techniques that are complex and often used to be performed by humans. AI can be used in techniques like advanced decision making, computer vision to identify obstacles, content recommendation based on algorithms and picture and video processing. Machine Learning (ML) is a subfield of AI, usually concentrating on handling large amounts of data, making predictions and learnings from this data. The performance of systems should improve while more data experience is built. (Helsingin Yliopisto & MinnaLearn, 2025). A good example of AI usage is delivery robot Starship Technologies has created. These robots can be used to deliver groceries, food and other small deliveries. The delivery robots are using cameras, ultrasonic sensors and machine learning to navigate on walkways and identifying obstacles and stop to red lights at a safe distance. This demonstrates good usage of advanced decision making and computer vision. They were also designed to look cute and easy to approach to avoid vandalism. (Starship, 2025).

There are several ways of using AI to improve defect report lifecycle. Using historical data can improve defect management strategies. Jahanshahi, H et al. (2022) explored defect report evolution by analysing open-source software defect reports. Understanding evolutionary behaviour can enhance the effectiveness of defect management and improve defect prioritizing strategies. It helps understanding how defect dependencies, number of defects and unresolved defects evolved over time. The different elements of past data can reveal the reason behind specific defect prioritization decisions, like why developer chose to resolve one defect over another. Severity and priority are very subjective, but it's possible to examine how these change over time from open defects to closed ones. This can provide insight into are they handled subjectively during triage.

Examining dependencies between different defects can give insights to what leads to delays in defect resolution. For example, blocking defects tends to have higher complexity than non-blocking defects, and usually take more time

to fix. Increase in blocking defects also can create difficulties in resource planning and therefore slow down the blocking defect fixing even more. Accumulating blocking defects may lead to decrease in software quality. (Jahanshahi, H et al. 2022). Detecting historical patterns can provide clear insight into several areas of defect management.

Machine learning can help correct descriptive quality of bug reports. Gromova, A. et al. (2019) introduced a prototype tool using machine learning that helped QA engineers evaluate and improve quality of defect report they've written. Quality of the defect report was evaluated based on several indicators in defect report description. Key indicators in this paper were probability of belonging to a specific area of testing, probability of defect being rejected, priority levels and probability of defect being fixed and time to resolving the defect. ML was applied to defect description for analysis and prediction of outcome based on these indicators.

Quality of defect reports could be improved using these metrics and by giving suggestions to QA engineers how they can improve the description of defect reports. The predicted indicators serve as a form of feedback to QA engineers. By analysing these probabilities, QA engineers can refine their defect reports to be more clear, actionable, accurate and comprehensive before submitting the report. The assessment that the tool provided of report quality based on the data can be particularly helpful for Junior QA Engineers. The tool can guide them improving their defect reports by including details that are statistically associated with well-received and efficiently resolved defect reports. Iterative process of modifying descriptions of AI-driven probability calculations helps QA Engineers to raise quality of their defect reports and can decrease the time to resolve the defect. (Gromova, A. et al. 2019).

Escaped defect analysis process usually involves inspecting defect reports to identify and analyse which ones are escaped defects. Defects that are not detected during test level it is supposed to be referred as escaped defect. Cruz, et al. (2023) discovered in their research that ML techniques can be applied to

automate and improve the process of escaped defect analysis. Escaped defect analysis can be time consuming and error prone process and therefore adding costs. ML models were trained with historical data of escaped defects to identify pattern in escaped defects. Data was used to prioritize manual inspection of escaped defects, prioritizing the most probable first.

Different ML algorithms have varying requirements for training and prediction. While some algorithms are noted to be faster, others might offer slightly better predictive performance. The choice of algorithm might involve a trade-off between the desired level of accuracy and the available resources and time constraints. (Cruz, et al. 2023).

As discussed earlier, well-written defect reports receive faster resolutions compared to ones that are written unclearly or missing information. AI and ML offer innovative solutions to automate quality checks, making the defect reporting process more efficient and accurate.

3.3 AI's Answer to Improving Quality of Defect Reports

While writing this thesis, I found that there was a limited amount of research papers published specifically just about the topic of how AI can improve defect report quality itself. So, I decided to ask this question using two different generative AI tools. Tools used were OpenAI ChatGTP-4.o mini and Google's Gemini Advanced 2.0 Flash.

Prompt provided was simple to ensure that the answer given would stay simple and short, as this was not the main objective of this thesis. Prompt given was: "How to improve bug reports with data and AI?". Complete answers of Gemini and ChatGPT can be found from Appendix 1 and 2.

Both answers were quite similar to other studies and references I found about the topic. Both responses focused on the question of how AI or AI and data together can improve defect reporting. I found Gemini's response more

complete than ChatGPT for a few reasons. Gemini answers in clear sentences, even when they were in bullet points. Gemini's answers were describing what AI can do and how it improves defect reporting. ChatGPT's answers were mostly describing what AI can do, and there was less of explaining how that improves defect reporting or add value compared to what a human QA Engineer could do. Responses focused more on bug report management and process of resolving defects than how defect report itself. In Gemini's answer there was also key technologies and benefits summarized. The essence of the answers was quite like each other, with only a few nuances. Overall, the answers were generally consistent with the information available on this topic.

Key takeaways how AI can enhance defect reporting were automatic data collection and analysis and assisting in writing the report. Collecting environment details, screen shots, user actions, crash reports and logs and adding them to defect reports automatically will reduce the time and improve the details added to defect reports. Creating summaries for complex defect reports help readers to grasp situations fast. Assisting in writing the report title and description can help make the report clearer and more consistent.

Biggest difference between ChatGTP and Gemini was image creation. Both were prompted with "Create an image of your answer illustrating benefits of using data and AI to improve bug reporting". Gemini did not create an image, ChatGTP did. ChatGTP stated that this image highlights AI-powered bug categorization, automated log analysis, predictive analytics, and developer collaboration with smart AI insights (figure 2).



Figure 2. ChatGTP created an image of how AI can improve data and defect reporting.

In the image created by ChatGTP (figure 2), there are typos in the text and some of the people are missing parts, such as hands and head. It is not easy to distinguish from this picture that it is trying to illustrate how AI can improve defect reporting and data. Gemini did not create image, just provided illustration and image ideas and what tools to use to create them.

In general, there is potential in AI tools, however, there is still a need for human experts in this process. Software is typically created for people like end-users and stakeholders. If input data or specifications are incomplete or ambiguous, output may not be what has been expected. Both people and AI can have biases that negatively affect the result.

4 The Human Factor in Defect Reporting

4.1 Effective Communication in Defect Reports

To be effective in communicating with developers and other stakeholders, understanding how they think will be helpful. Knowing and learning how developers and other stakeholders think will improve the communication. (Bach, J. et al. 2002). This also improves the effectiveness of defect report as a communication tool. Different roles in the project tend to focus on their core area. For example, project manager might be more interested how does the defect impact on timeline and resources, compared to developer who might look more that information provided is complete and steps to reproduce problem are clearly stated. Understanding different perspectives and ensuring that relevant information to all stakeholders is provided in the defect report makes it more effective. This makes it important to use clear and concise communication in defect reports and avoid ambiguity. “A picture is worth thousand words” saying goes well with effective communication in defect reports. Images, screenshots and screen recordings create clarity and understanding to the reader with one glance. They also can help understand the issue from end-users point more clearly than words. Only necessary details should be added to avoid overloading reader with information and reduce redundancy.

Discussion in person provides valuable information in complex issues. Clarifying questions help understand issue in hand and help provide necessary information to all parties. Active listening will help understand situation better.

Quality matters across the project and development phases and should be a joint effort of a project team.

4.2 Collaboration and Feedback in Defect Reporting

Collaboration and feedback play an important role in the defect reporting process. Asking relevant questions of the work product will help understand

software under testing, and it will also help forming understanding with people working on it. Largest group to collaborate with defect reports tends to be developers. Creating a trust-based relationship with developers will be more effective than being on the opposite sides. Respecting their time and expertise will help with this relationship. Finding out what kind of feedback they want and then giving it will improve collaboration. When giving feedback, concentrate on the work, not the person delivering the work. (Bach, J. et al. 2002). Constructive feedback is better received when given directly to the person in question, and positive feedback has greater impact when given in a group. (Ahola, T et al 2002).

Constructive criticism given well is a skill that is difficult to master. Problems may come from system errors or human actions, and it's important to distinguish between these causes before determining how to address the issue effectively. Overall, it's more important to focus on solutions rather than blame. People have an easier time taking in wishes and suggestions of how they are expected to behave, rather than criticize what they are doing wrong. Usually, they tend to know themselves too how they are behaving wrong but may not be aware what they should do instead. This in mind, it can be more constructive to suggest a new way to work rather than criticize what they are doing wrong. It will be more effective and motivating when discussing what are the benefits of this new way of working, creating a plan that will be taken into action and then giving positive feedback of trying the new ways of working. This method can be more effective and positive than analyzing a problem when it's caused by human actions. Analyzing human actions can just lead to vicious circle (figure 3). (Ahola, T et al 2002).



Figure 3. Image illustrating circle of analysing human actions.

While problems need solving, goals provide direction and motivation. Keeping this in mind, it's often more effective to establish goals and work towards them rather than focusing on assigning blame when addressing human related issues. (Ahola, T et al 2002). For example, in situations where communication issues arise, it can be more beneficial to create a goal of improved communication and outline steps to achieve it, instead of analysing who is communicating poorly and why.

Communicating is essential in defect reporting ensure a shared understanding of the reported issue and to build consensus on its impact, priority, and severity. So, if defect reports are often missing reproduction steps, a constructive goal could be to that all defect reports should have reproduction steps. One way to achieve this is to add a "Steps to Reproduce" header to the defect report

template or base used. This approach provides a clear path for improvement and encourage solution-oriented mindset.

4.3 The Role of Empathy in Defect Reporting

Defect report can essentially be taken as criticism of someone's work, so it's vital to deliver this news with sensitivity, empathy and diplomacy. Objective of defect report is supporting improvement and raising quality, not to assign blame.

One cornerstone of a good working environment is caring. Caring includes appreciation, curiosity towards other people's interests, encouraging, comforting, teaching and helping others. Caring plays a vital role in good workplace environment. Simple things like saying hello, being interest in what other team members are doing and how they are doing will help creating a caring environment. Helping others is one of the most important secrets of creating a good working environment. Helping others creates a sense of caring to the person who is helped, and an opportunity to show appreciation towards the other person. It also helps improve efficiency at work. (Ahola, T et al 2002).

Collective empathy in software development plays large role in efficient and good working environment. Collective empathy is shared understanding of the team members thoughts, believes, actions and feelings. It goes over the individual empathy and is present in team level, understanding each other needs and pressure points. (Akgün, A et al. 2015).

Akgün, A et al. (2015) discovered in their study that collective empathy has influence and consequences in software development project. This study shows that collective empathy within the team has a positive influence on their behavior. Mutual trust, recognition and appreciation towards others' work will increase the productivity and learning of the entire team. Empathy provides an environment where teams can excel in discovering and fixing software

problems. This leads to increased productivity and more efficient learning environment, benefiting the defect reporting process.

Promoting meaningful discussions that goes beyond the task-related discussions (Akgün, A et al. 2015).

This all has clear implications for defect reporting and its handling. Empathy will help create culture of continuous improvement. The primary goal is to foster a culture where defects are seen as opportunities to enhance software quality rather than failures of people, so empathy is essential.

Empathy plays a crucial role in changing defect reporting from potentially negative process into positive driver of software quality. By encouraging trust, promoting constructive communication and encouraging collaboration, empathy creates a foundation for teams to thrive as they discover and resolve defects.

5 Conclusion

Defect reports are the cornerstone of effective software development. Well written, clear and actionable defect report has great impact on software quality and it lays a foundation for efficient software development. Defect reports are not only documenting what part of the software does not meet expectations, but they are also great tool in process improvements. Defect reports include data that can be analysed to create better test coverage process improvements, and even train AI models. Defect reports can be used to gather insights and information of the qualitative state of the software, and based on that, create better processes to prevent defects in these areas in future. Preventing defects will help raise quality of the product but also save time and money in future.

In the end, a good defect report is based on fairly simple things that can be difficult to master. Clear, concise and detailed defect reports that will convey the

issue and its severity in a language which is understandable to all stakeholders and developers is crucial for correct prioritizing and fast resolution.

Through automation, predictive analysis and raising descriptive quality AI has a role in all this by enhancing and speeding up the defect lifecycle. Correctly used AI can be a helpful tool that powers up quality assurance. It can help both increase the level of quality and save time used. Using data from defect lifecycle, ML becomes powerful tool. Processes can be improved with analysing delays in defect resolution. Improving descriptive quality of defect report with suggestions given to QA Engineers. Part of the escaped defect analysis can be automated with ML. This all can save time and resources. But not to forget, quality of input data and specifications has high correlation to result, and its always important to check the output of AI work products. While AI offers automation and efficiency, human skills in are still needed in ensuring quality in software development.

As technologies advance, the human factor still remains. Good communication is a key for successful software development project. Communication, collaboration and empathy are crucial for effective defect reporting and defect resolution. Processes can be refined to maximum, but it won't be beneficial if people are not on board with them and execute the processes.

Topics in this thesis covered what makes a good defect report, how it can be improved and how that can contribute to overall software quality, and key factors how to improve this quality. Looking ahead, role of QA engineers will continue to emphasis strategic thinking, complex problem solving and collaboration with AI systems. This can present a challenge how to cope with growing difficulty level. The next step would be to examine how use of AI powered tools can enhance job satisfaction and wellbeing and how this improvement would affect the overall software quality.

References

Ahola, T. Furman, B. (2002) Työpaikan hyvä henki ja kuinka se tehdään. Tampere, Tammer-Paino Oy.

Akgün A., Keskin H., Cebecioglu Y., Dogan D. (2015) Antecedents and consequences of collective empathy in software development project teams. Available at: <https://www.sciencedirect.com/science/article/pii/S037872061400144X> (Accessed: March 12, 2025).

Bach, J. Kaner, C. Pettichord, B. (2002) Lessons Learned in Software Testing, A Context-Driven Approach. New York, USA. Wiley & Sons, Inc.

CQI (2023) Driving sustainable IT with quality assurance. Available at: <https://www.quality.org/article/driving-sustainable-it-quality-assurance> (Accessed March 12, 2025).

Bettenburg, N., Just, S., Schröter, A., Weiss, C., Premraj, R., & Zimmermann, T. (2008). What makes a good bug report? SIGSOFT 2008/FSE-16, Atlanta, Georgia, USA.

Cruz, P. H. A., Filho, A. A. P., Moreira, P. R. S., Mota, A. C., Nascimento, L. P. G., Oliveira, D. C. C. A. de and Prudêncio, R. B. C. (2023) Machine Learning Techniques for Escaped Defect Analysis in Software Testing. 8th Brazilian Symposium on Systematic and Automated Software Testing Campo Grande, MS, Brazil. ACM. Available at: <https://doi.org/10.1145/3624032.3624039> (Accessed March 24, 2025)

GeeksforGeeks. (2023) 'Differences between Software Testing and Quality Assurance', *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/software-testing-vs-quality-assurance/> (Accessed: March 4, 2025).

Gromova, A., Pavlov, S., Itkin, I. and Korovayev, A. "Raising the Quality of Bug Reports by Predicting Software Defect Indicators", 2019 IEEE 19th International Conference on Software Quality, Reliability and Security Companion [QRS-C], Sofia, Bulgaria, 2019. Available at: <https://ieeexplore-ieee-org.ezproxy.metropolia.fi/stamp/stamp.jsp?tp=&arnumber=8859465> (Accessed: March 5, 2025).

Helsingin Yliopisto, MinnaLearn. Elements Of AI, 2025. Available at: <https://course.elementsofai.com/> (Accessed March 28, 2025).

International Software Testing Qualifications Board [ISTQB]. (2024). Certified Tester Foundation Level Syllabus v4.0.1. Available at: https://www.istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf (Accessed: March 4, 2025).

International Software Testing Qualifications Board [ISTQB CTAL-TM]. (2024). Certified Tester Advanced Level Test Management Syllabus. Available at: https://www.istqb.org/wp-content/uploads/2024/11/ISTQB_CTAL-TM_Syllabus_v3.0_zKjKsaN.pdf (Accessed: March 17, 2025).

Jahanshahi, H., Cevik, M., Navas-Sú, J., Başar, A., González-Torres, A. (2022) Wayback machine: A tool to capture the evolutionary behavior of the bug reports and their triage process in open-source software systems, Journal of Systems and Software. Available at: <https://www.sciencedirect.com.ezproxy.metropolia.fi/science/article/pii/S0164121222000565> (Accessed: March 5, 2025).

Nyman M., Tivia verkkouutiset (2024). Available at: <https://www.tivi.fi/uutiset/s-pankki-ei-uskonut-teinin-varoituksia-haavoittuvuudesta-joutui-karsimaan-miljoonavahingon/119a41f4-8c56-463a-af7f-64e26f7f7f67> (Accessed: March 5, 2025).

Son, H. (2024). How to improve bug reporting & management for better results. Available at: <https://www.testrail.com/blog/improve-bug-reporting/> (Accessed: March 4, 2025).

Starship. Available at: <https://www.starship.xyz/our-robots/> (Accessed March 28, 2025).

Vala Group Oy (2024) How to automate the daily routine of QA engineers – An AI based solution example. Available at: <https://www.valagroup.com/blog/how-to-automate-the-daily-routine-of-qa-engineers-an-ai-based-solution-example/> (Accessed: March 5 2025).

Discussion with Google Gemini Advanced 2.0. Flash

The integration of data and AI is significantly transforming how bug reports are handled, leading to more efficient and effective software development. Here's a breakdown of how this improvement is happening:

How Data and AI Improve Bug Reports:

- **Automated Bug Detection:**

- AI algorithms can analyze code patterns and historical data to identify potential bugs in real-time. This reduces the reliance on manual testing and catches errors that might be missed by humans.
- Predictive analytics can forecast areas where bugs are likely to occur, allowing developers to focus their testing efforts.

- **Enhanced Bug Reporting:**

- AI-powered tools can automatically capture relevant data, such as system logs, screenshots, and user actions, to provide detailed bug reports.
- Natural Language Processing (NLP) can analyze bug descriptions and categorize them based on severity, priority, and affected components.
- AI can assist in writing more clear and concise bug reports, by helping to generate the title, description, and the steps to reproduce the bug.

- **Improved Bug Triage and Prioritization:**

- AI can analyze historical bug data to identify patterns and trends, helping to prioritize bugs based on their impact and frequency.

- Machine learning algorithms can learn to recognize duplicate bug reports, reducing redundant work.
- **Faster Bug Resolution:**
 - AI can provide code suggestions and auto-correction, simplifying the debugging process.
 - By providing more detailed and accurate bug reports, AI helps developers quickly understand and resolve issues.

Key Technologies and Tools:

- **Machine Learning (ML):** Used for pattern recognition, predictive analytics, and automated classification.
- **Natural Language Processing (NLP):** Used for analyzing and understanding bug descriptions and generating reports.
- **Automated Testing Frameworks:** Used to execute tests and automatically generate bug reports.
- **AI-powered Bug Tracking Systems:** Systems that incorporate AI to automate various aspects of bug management.

Benefits:

- **Increased Efficiency:** Automation reduces the time and effort required for bug detection and reporting.
- **Improved Accuracy:** AI minimizes human error and provides more comprehensive bug reports.
- **Faster Resolution:** Detailed and accurate reports, along with AI-powered debugging tools, accelerate the bug-fixing process.

- **Enhanced Software Quality:** Early detection and efficient resolution of bugs lead to higher-quality software.

In essence, data and AI are making bug reporting a more data-driven and automated process, leading to significant improvements in software development efficiency and quality.

.

Discussion with ChatGTP-4.o mini

Improving bug reports with **data and AI** can significantly enhance issue resolution efficiency, reduce debugging time, and improve software quality.

Here are key strategies:

- **1. Automated Data Collection**
 - **Log Analysis:** Automatically attach relevant logs, stack traces, and system events to bug reports.
 - **User Session Replay:** Use AI to analyze user actions leading up to the bug.
 - **Crash Reports:** Integrate automated crash reporting tools (e.g., Sentry, Firebase Crashlytics).
 - **Environment Details:** Include OS, browser, app version, device model, and network conditions.
- **2. AI-Powered Bug Categorization & Prioritization**
 - **Text Analysis:** Use Natural Language Processing (NLP) to classify bugs based on descriptions.
 - **Historical Data:** Train AI models on past bug reports to predict severity and priority.
 - **Duplicate Detection:** AI can detect similar issues already reported to avoid redundancy.
- **3. Smart Suggestions & Auto-Resolution**
 - **Suggested Fixes:** AI can recommend probable solutions based on similar past bugs.

- **Code Impact Analysis:** AI can analyze code changes to suggest potential root causes.
- **Automated Debugging:** AI can generate test cases to reproduce and verify bugs.
- **4. Predictive Analytics & Anomaly Detection**
- **Trend Analysis:** Identify patterns in bug reports (e.g., specific features prone to issues).
- **Early Warnings:** AI can flag potential issues before they escalate by analyzing anomalies in user behavior or system logs.
- **5. Improved Developer Experience**
- **AI-Generated Summaries:** AI can summarize long bug reports to extract key details.
- **Conversational Bug Reporting:** AI-powered chatbots can guide users through structured bug reporting.
- **Automated Ticket Assignment:** AI can route bug reports to the most relevant team or developer.

Would you like recommendations on specific AI tools or platforms to implement these strategies?